iacc instituto profesional

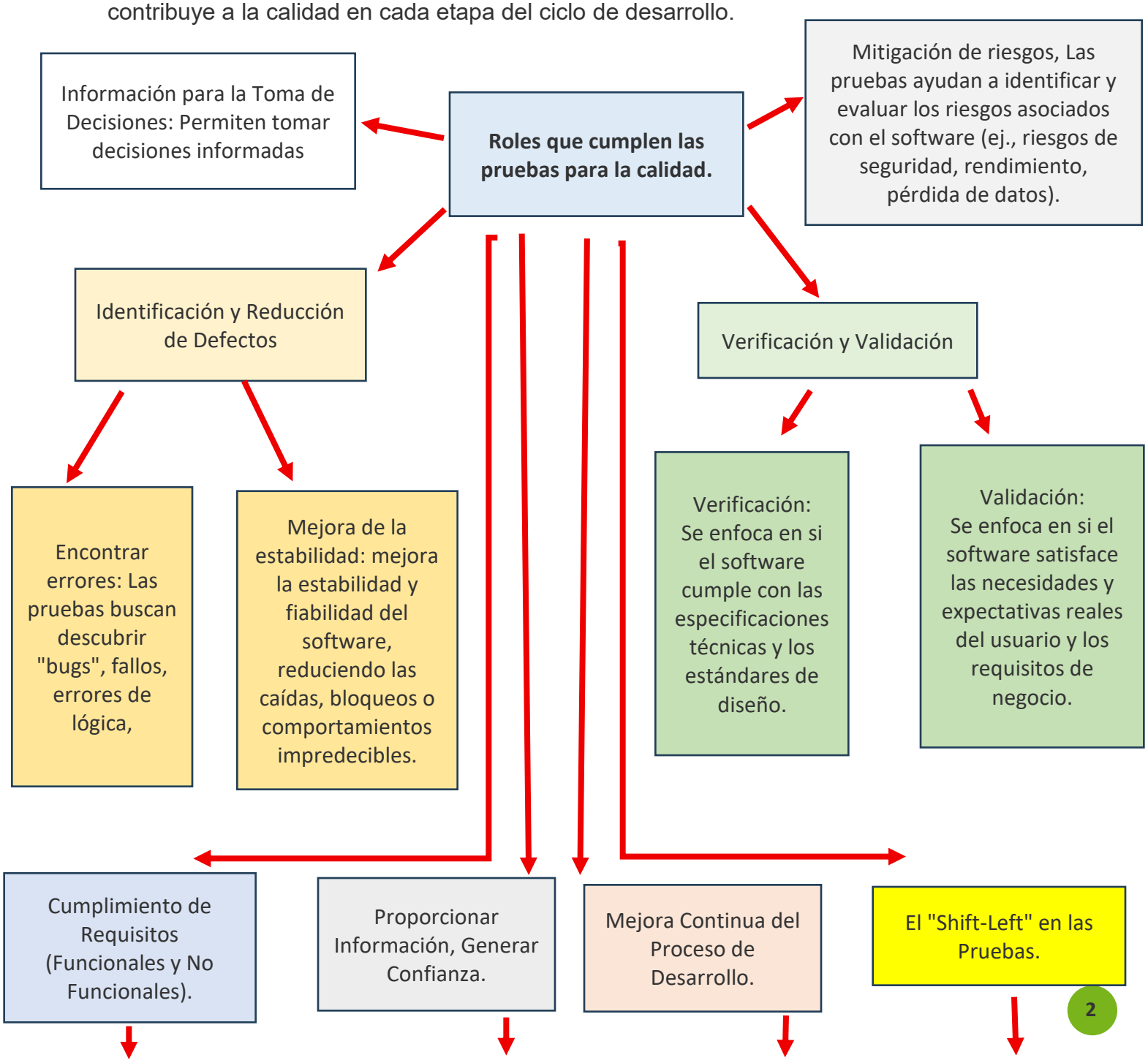
EVALUACIÓN

Herramientas de QA
Semana 2

Amparo Valdivia
27, junio de 2025
Técnico en Análisis y Programación.

DESARROLLO:

Para el desarrollo de una aplicación móvil para una empresa de entrega de alimentos, el rol de las pruebas (o "testing") en el desarrollo de software es absolutamente fundamental y se caracteriza por ser dependiente del objetivo y los requisitos que debe poseer el software, para garantizar la calidad del producto final. Es un proceso integral que contribuye a la calidad en cada etapa del ciclo de desarrollo.



Requisitos Funcionales: Las pruebas verifican que cada característica o función del software se comporte como se espera.

Requisitos No Funcionales:

- **Rendimiento:** ¿Es rápido? ¿Soporta la carga de usuarios esperada?
- **Seguridad:** ¿Está protegido contra ataques? ¿Los datos sensibles están seguros?
- **Usabilidad:** ¿Es fácil de aprender y usar para el usuario final?
- **Fiabilidad:** ¿Funciona de manera consistente a lo largo del tiempo?
- **Mantenibilidad:** ¿Es fácil de modificar o corregir en el futuro?
- **Portabilidad:** ¿Funciona en diferentes entornos o dispositivos?

Un software que ha pasado por un riguroso proceso de pruebas y ha demostrado ser estable y fiable, genera confianza no solo en el equipo de desarrollo, sino también en los usuarios finales y los clientes.

Las pruebas no solo evalúan el producto, sino también el proceso que lo creó, Esta retroalimentación es invaluable para mejorar futuras iteraciones del software y los procesos de desarrollo en general.

(mover a la izquierda), las pruebas se integran desde etapas tempranas, **Revisión de requisitos:** Asegurar que los requisitos sean claros, completos y testeables. **Revisión de diseño:** Identificar posibles fallos de arquitectura o diseño antes de que se escriba una sola línea de código. **Pruebas unitarias:** Los propios desarrolladores prueban módulos individuales de código. **Integración continua:** Las pruebas se ejecutan automáticamente cada vez que se integra nuevo código.

•Para definir los objetivos y requisitos de la aplicación móvil, debemos comprender varios factores:

Con el propósito de mejorar y agilizar su servicio, el software debe contener los objetivos funcionales y los no funcionales

Requisitos Funcionales (RF): Describen **lo que el sistema DEBE HACER**. Son las acciones o funcionalidades específicas que el usuario espera del sistema.	**Requisitos No Funcionales (RNF):** Describen **CÓMO el sistema DEBE SER**. Se refieren a la calidad, rendimiento, seguridad, usabilidad, eficiencia, etc.
"manejar stock disponible de alimentos actualizado", Esto es una **funcionalidad específica**. El sistema debe poder rastrear y mostrar el stock en tiempo real. Es una acción concreta que el software debe realizar para cumplir su propósito.	"reducir las preguntas para que el usuario se registre y hacer más amigable el proceso", Se refiere a la **usabilidad** del sistema, es una característica de calidad sobre cómo debe ser el proceso de registro para que sea fácil y agradable para el usuario.
Que el software muestre los precios ordenados de menor precio a mayor precio, Es una acción concreta que el sistema realiza (la función de ordenar).	"organizar mejor la información, del sitio web", se relaciona con la **usabilidad** y la **arquitectura de la información**. Implica que el diseño y la presentación de los datos deben ser claros e intuitivos, lo cual es una cualidad del sistema, no una acción.
Que el usuario pueda agregar alimentos en una canasta, el usuario puede realizar la	"agilizar los procesos con las bases de datos", Esto apunta a la **eficiencia** y el

acción de agregar dentro de la aplicación, y que el sistema debe procesar.	**rendimiento** del sistema. No es una funcionalidad visible para el usuario, sino una expectativa sobre la rapidez y optimización de las operaciones internas del software con la base de datos.
Mostrar al usuario la cantidad de productos que eligió y el total de la compra, es una parte fundamental del flujo de compra y le permite al usuario tomar decisiones informadas antes de finalizar su pedido.	"al programar el html, que el css quede bien hecho y no se descuadre cuando el cliente lo vea ya sea en un móvil o Tablet", Se refiere a la **compatibilidad**, **diseño responsivo** y **usabilidad/experiencia de usuario**. Es una cualidad sobre cómo debe verse la interfaz en diferentes dispositivos, asegurando una buena experiencia visual, no una función que el usuario interactúa directamente.
Funciones, acciones entre usuario y la API	características de calidad, atributos, restricciones o criterios de desempeño de la API.

Los requisitos funcionales y los no funcionales tienen pruebas necesarias para verificar los procesos, como es mencionado en el material de profundización del ramo semana 2, se pueden utilizar herramientas de automatización de pruebas que facilitan la ejecución de casos de prueba repetitivos y agilizan el proceso de prueba, mientras que el material interactivo nos profundiza que ‘Cada herramienta posee características y enfoques

particulares. La elección de la herramienta adecuada dependerá de los requisitos del proyecto, la naturaleza de la aplicación o sistema a evaluar y las necesidades específicas del equipo de pruebas de seguridad':

Pruebas funcionales:	**Pruebas no funcionales:**
Verifican que cada característica o función del software se comporte como se espera, basándose en los requisitos funcionales. Se enfocan en lo que hace. Aseguran que el software entregue las funcionalidades acordadas y complazca las necesidades del usuario.	**Se refieren a las características del sistema relacionadas con el rendimiento, la seguridad, la usabilidad, la fiabilidad, etc.** **No describen una funcionalidad en sí misma, sino atributos de calidad.**
Para verificar la capacidad de realizar pedidos, el usuario puede efectivamente generar un pedido, y realizar el pago exitosamente. Proporcionar notificaciones en tiempo real a los usuarios sobre entregas es un requisito **funcional**. **Pruebas de casos de uso.** Verifican que el sistema funcione correctamente para escenarios de interacción completos. 1.- El usuario abre la aplicación y se inicia sesión con credenciales válidas. 2.- El usuario ve los productos.	**Pruebas de rendimiento:** Evalúan el desempeño del software bajo diferentes condiciones, como, por ejemplo, el límite de usuarios simultáneos que puede soportar la app durante un evento como promociones, así se puede planificar el crecimiento a largo plazo. Esta prueba es necesaria en el software de entrega de alimentos, ya que este debe soportar picks de ventas y pedidos simultáneos, sin perder la calidad y la rapidez de las operaciones.

3.- Se agregan los productos al carrito. 4.- Revisa el carrito, verifica el total de la compra (incluyendo impuestos y costo de envío). 5.- Procede al pago, selecciona "Tarjeta de Crédito" como método. 6.- Ingresa los datos de la tarjeta de crédito válidos y la dirección de entrega guardada. 7.- Confirma el pedido. 8.- Verifica que se muestra una pantalla de confirmación con el número de pedido, la dirección y el tiempo estimado de entrega. 9.-Cierra sesión. El pedido se registra correctamente en el sistema, se realiza el cargo a la tarjeta, el usuario recibe una confirmación, y el stock de los productos se actualiza.	En las pruebas de rendimiento se puede verificar que las notificaciones lleguen a los usuarios en un tiempo aceptable, se puede probar simular un gran número de entrega que desencadenen notificaciones simultáneamente, la cantidad de compradores que puede soportar y ver si el sistema puede aumentar su capacidad de envío de notificaciones, y poder recomendar aumentar la capacidad de los equipos, si fuese necesario.
Pruebas de interfaz de usuario (UI). Se enfocan en la parte visual de la aplicación, asegurando que todos los elementos gráficos (botones, íconos, textos, imágenes, campos de entrada) se muestren correctamente, estén alineados,	**Pruebas de seguridad:** Estas pruebas se realizan al manejar datos sensibles de pago y usuario, para garantizar la protección de información personal y financiera, la revisión se hace produciendo errores para verificar que se ocupen los filtros adecuados.

que concuerden con el diseño especificado y sean interactivos. Proporcionar notificaciones en tiempo real a los usuarios sobre entregas. Pruebas de funcionalidad básica. Son un conjunto de pruebas rápidas y de alto nivel que se realizan en una nueva construcción (build) del software para asegurar que las funcionalidades más críticas y principales están operativas antes de proceder con pruebas más exhaustivas. Pruebas de validación de datos. Verifican que el sistema acepta datos válidos y rechaza datos inválidos o incompletos en los campos de entrada, mostrando mensajes de error apropiados.	Esta prueba es relevante porque pueden peligrar las transacciones, el robo de datos, etc. Debido a esto debe ser una prueba exhaustiva. Pruebas para encontrar debilidades del software Identificación de riesgos. • Diseño de casos de prueba. Ven vulnerabilidades. • Pruebas de penetración, simula ataques reales. • Análisis estático, revisión de código que puedan comprometer la seguridad. • Dinámico, evalúa su comportamiento frente a diferentes escenarios y ataques. • Pruebas de configuración. Evalúan el entorno del software como el servidor o la base de datos. • Evaluación de riesgos y reporte de resultados.

Técnicas de Diseño de Pruebas y Enfoques de Testing, funcionales. herramientas que se utilizan para definir pruebas que verifican el sistema. **Particiones de equivalencia.** Es una técnica de prueba de caja negra significa que el código no se revisa solo la funcionalidad de este, donde se dividen los datos de entrada de una función ósea los parámetros, en "clases de equivalencia". **Utilidad:** Reduce el número de pruebas. **Valores límite.** Para cada límite, se prueban los valores justo dentro del límite, en el límite y justo fuera del límite. Ejemplo "Cantidad de un artículo" a añadir al carrito, con un stock disponible de 1 a10 unidades o >0 y <10. **Análisis de causa-efecto.** Es una técnica de diseño de pruebas que se utiliza para ver casos que puedan suceder imitando situaciones complejas donde múltiples condiciones de entrada (causas) influyen en múltiples resultados	**Pruebas de usabilidad:** Se realizan para, para asegurar una experiencia del usuario fluida. Es clave para el software ya que, sin buena usabilidad, el cliente no usará la app. "la interfaz de usuario debe ser intuitiva" es fundamental para la experiencia del usuario. Para probar esto, se realizan las pruebas de usabilidad (Usability Testing). Como ejemplo están las pruebas de usabilidad moderada que es cuando se invitan a usuarios y se les pide que interactúen con la aplicación y vayan explicando cómo se sienten ocupándola en voz alta, así se puede ir documentando la experiencia en vivo. El moderador puede identificar directamente dónde los usuarios dudan, se confunden, buscan ayuda o no encuentran lo que esperan. Las pruebas de usabilidad no moderadas sin un moderador presente, es cuando los usuarios reciben las instrucciones de las tareas y graban su pantalla y audio,

(efectos). Ejemplo Aplicación de descuentos en el total del pedido. **Tablas de decisión.** Es una forma de representar las relaciones entre condiciones y acciones, ejemplo Procesamiento de pedido y notificaciones. 1: Stock Disponible (Sí/No) 2: Pago Autorizado (Sí/No) 3: Dirección de Entrega Válida (Sí/No) **Pruebas de transición de estados.** Es una técnica que cambia el comportamiento del sistema como una serie de estados y las transiciones (eventos) que hacen que el sistema pase de un estado a otro, ejemplo, Ciclo de vida del estado de un pedido, Pendiente. Confirmado. Preparando. En camino.	expresandose en voz alta de forma remota utilizando herramientas especializadas, como por ejemplo, Hotjar Engage, UserTesting, Maze).

Pruebas exploratorias. El tester diseña y ejecuta pruebas simultáneamente, sin scripts predefinidos, utilizando su conocimiento del producto, su experiencia, ejemplo, Explorar la experiencia de un usuario nuevo que acaba de instalar la app.	
Pruebas unitarias: Se enfocan en verificar el funcionamiento de unidades individuales de código, por ejemplo, probar función por función, como: def agregar(algo): **hacer un script que pruebe esa función, y revise si están los parámetros en la base de datos.** **Importancia:** Detectan errores de forma temprana, facilitan la depuración.	**Pruebas de fiabilidad:** Por ejemplo, se utilizan para probar la recuperación ante fallos del sistema de pedidos en caso de caída del servidor o base de datos. La app esté disponible constantemente, lo cual es crítico en un servicio de entrega. la recuperación ante fallos durante el proceso de pago, donde se simula una caída o interrupción de la conexión a la base de datos, la aplicación debe ser capaz de retomar el proceso o revertir la transacción sin perder datos del usuario o del pedido una vez que la conexión se restablece.
Pruebas de integración: Evalúan la interacción entre componentes o módulos,	**Pruebas de mantenibilidad:** Se realizan cuando ha habido cambios en la

ejemplo, la interacción entre el módulo de carrito de compras y el módulo de inventario al añadir un producto; la conexión entre la app móvil y la API de pedidos. **Importancia:** Aseguran que los componentes trabajen juntos sin problemas.	arquitectura y es necesario verificar el sistema, en revisiones de código también.
Pruebas de sistema: Prueban el software completo, ejemplo, un usuario registrado que navega, añade productos, paga, y recibe confirmación. **Importancia:** Valida el comportamiento de extremo a extremo del sistema. Estas pruebas son esenciales para el software de la aplicación móvil para una empresa de entrega de alimentos, para la revisión de la calidad del flujo de pedido de todo el software, la gestión de la canasta, el pago y la visualización del stock. Esto simula la experiencia real del usuario.	**Pruebas de portabilidad:** Si se planea desplegar la aplicación en diferentes plataformas de nube, diferentes y nuevas tecnologías móviles, enviando notificaciones a diferentes modelos de teléfonos o tablets, esta debe llegar de igual forma.
• **Pruebas de regresión:** Se ejecutan después de cualquier cambio en el código, nuevas funcionalidades,	

correcciones, para asegurar que los cambios no hayan introducido nuevos defectos o roto funcionalidades existentes, ejemplo, si se añade una nueva forma de pago, se vuelve a probar que las formas de pago antiguas siguen funcionando, y que el registro de usuario no se dañó. **Importancia:** Mantienen la estabilidad del software a medida que evoluciona. Esta prueba también es muy importante ya que un software de venta de alimentos tiene cambios constantes como nuevas opciones de pago, o nuevos productos.	
Pruebas de aceptación del usuario: realizadas por usuarios finales para validar el cumplimiento de los requisitos. Según Fernández Riera, M. (2022). Las pruebas de aceptación se dividen en dos grandes grupos: Las pruebas alfa: se llevan a cabo, por un cliente, en el lugar de desarrollo. Se usa el software de forma natural con el desarrollador como observador del usuario	

y registrando los errores y problemas de uso. Las pruebas beta: se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. Generalmente, se llevan a cabo, justo antes del lanzamiento y puesta en producción de una aplicación software.	

Como es mencionado en el texto de Fernández Riera, M. (2022). Puesta en producción segura, “Cada fase en el desarrollo de pruebas está asociada a una etapa de la construcción de software, motivo por el cual, aunque no es estrictamente necesario que ambas fases trabajen simultáneamente, deberán hacerlo en colaboración constante”.

- Observando las pruebas y los procesos, creo que las pruebas funcionales preceden lógicamente a las pruebas de usabilidad, por ejemplo,

Si el botón "Agregar al carrito" no hace nada al hacer clic que es una falla funcional, entonces no importa si es intuitivo el diseño del botón o si es fácil encontrarlo; el usuario no podrá completar la acción.

- Entonces la usabilidad que es no funcional se vuelve irrelevante, por otro lado, las pruebas pueden unirse ya que, si una funcionalidad existe, su usabilidad determina si es fácil, eficiente y satisfactorio para el usuario realizar esa función, como por ejemplo el usuario puede filtrar productos por categoría, la usabilidad facilita lo funcional.

Herramientas para Pruebas de Rendimiento:

JMeter (Apache JMeter):

- **Finalidad/Utilidad:** Herramienta de código abierto, sirve para pruebas de carga y estrés.
- Puede simular un gran número de usuarios concurrentes enviando solicitudes a la API de la aplicación móvil.
- Permite medir el tiempo de respuesta, el rendimiento de la base de datos y el servidor.
- Graba los escenarios del usuario como inicio de sesión, búsqueda de alimentos, añadir los productos al carrito, pago.
- Ejecuta la prueba y analiza los informes de rendimiento.

LoadRunner (Micro Focus LoadRunner):

- **Finalidad/Utilidad:** Herramienta comercial de nivel empresarial para pruebas de rendimiento.
- Soporta una amplia variedad de protocolos y entornos.
- Similar a JMeter, con capacidades más avanzadas para simular entornos complejos y generar informes detallados.

Artillery.io:

- **Finalidad/Utilidad:** Herramienta basada en Node.js, para pruebas de carga y estrés de APIs.
- Ligera y fácil de integrar en flujos de CI/CD.
- Se escriben scripts de prueba en YAML o JavaScript para definir los escenarios de carga.

Herramientas para Pruebas de Seguridad:

OWASP ZAP (Zed Attack Proxy):

- **Finalidad/Utilidad:** Herramienta de código abierto para pruebas de penetración y escaneo de vulnerabilidades.
- Se pueden poner las reglas de seguridad.
- Permite encontrar vulnerabilidades comunes (ej. inyección SQL, XSS) en aplicaciones web y APIs. Puede interceptar y modificar el tráfico entre la app móvil y el servidor.
- Escaneo activo (ZAP ataca la aplicación automáticamente) y escaneo pasivo (ZAP analiza el tráfico mientras el tester interactúa manualmente).

Burp Suite (PortSwigger Burp Suite):

- **Finalidad/Utilidad:** Suite de herramientas comerciales y versión gratuita (Community Edition) para pruebas de seguridad web.
- Es un proxy interceptor para analizar, modificar y repetir solicitudes HTTP/S, esencial para probar la seguridad de las APIs REST/GraphQL que usa la app móvil.
- Interceptación de tráfico, análisis de solicitudes y respuestas, fuzzing de parámetros, etc.

SonarQube:

- **Finalidad/Utilidad:** Plataforma de código abierto para el análisis estático de calidad de código y seguridad.
- Identifica "code smells", bugs y vulnerabilidades de seguridad en el código fuente de la app móvil y del backend/API.

- Se integra en el pipeline de desarrollo (CI/CD) y analiza el código automáticamente en cada commit o build.

Herramientas para Pruebas de Usabilidad:

Lookback:

- **Finalidad/Utilidad:** Herramienta para grabar sesiones de pruebas de usabilidad.
- Incluyendo la pantalla del usuario, su cara y su voz mientras interactúan con la aplicación móvil.
- Permite tanto pruebas moderadas como no moderadas.
- Se puede invitar a participantes, proporcionarles tareas, y luego analizar las grabaciones para identificar puntos de confusión o dificultad.

Definición Clara de Objetivos y Requisitos:

- ✓ Antes de cualquier prueba, se debe tener una comprensión de qué se va a probar y por qué.

Elaboración de un Plan de Pruebas Detallado:

- ✓ Se define el alcance,
- ✓ Los recursos,
- ✓ El cronograma,
- ✓ Los criterios de entrada y salida,
- ✓ Los entornos de prueba y las métricas.

Diseño de Casos de Prueba Rigurosos:

- ✓ Utilizando las técnicas mencionadas, se crean casos de prueba detallados y replicables para cubrir todas las funcionalidades y escenarios de calidad.

Preparación del Entorno de Pruebas:

- ✓ Configurar dispositivos (móviles reales, emuladores/simuladores), versiones de SO, redes, datos de prueba, y acceso a APIs externas.

Ejecución Metódica de Pruebas:

- ✓ Seguir los casos de prueba,
- ✓ Registrar resultados (positivo funciona, negativo sigue fallando),
- ✓ Documentar cualquier desviación o error.

Gestión de Defectos Eficiente:

- ✓ Registrar errores en un sistema de seguimiento, asignar prioridades, dar tareas a los desarrolladores y seguimiento hasta su término.

Automatización de Pruebas:

- ✓ Identificar pruebas repetitivas (regresión, unitarias, algunas de integración), automatizarlas para ejecutarlas.
- ✓ Se pueden utilizar herramientas como Selenium (para web/navegador), Appium (para móvil), Cypress, Playwright, XCUITest (iOS), Espresso (Android).

Informes y Métricas:

- ✓ Generar informes de estado de pruebas, para la toma de decisiones.

Colaboración Continua:

- ✓ Mantener una comunicación fluida entre el equipo de QA, desarrollo, y otros dptos. durante todo el ciclo de vida del desarrollo.

"Shift-Left" y Pruebas Tempranas:

- ✓ Integrar las actividades de prueba desde las primeras etapas (revisión de requisitos y diseño) para identificar y corregir problemas cuando son menos costosos.

Como conclusión pude observar que el proceso de las pruebas asegura la calidad en cada etapa, en las pruebas funcionales, desde las Unitarias que validan cada línea de código de un cálculo de descuento, hasta las de Sistema que simulan un pedido completo, y las de aceptación con usuarios reales, garantizan que la aplicación funcione exactamente como los clientes esperan, mientras que las pruebas no funcionales, como las de Rendimiento, Seguridad, Usabilidad, Fiabilidad, Mantenibilidad y Portabilidad, se aseguran de que la aplicación “Como lo hace” calidad de desempeño, de la manera mas eficiente.

Al aplicar técnicas de diseño como las Particiones de Equivalencia y los Valores Límite, y al usar Tablas de Decisión para las reglas complejas, se están construyendo casos de prueba más exhaustivos que aumentan la probabilidad de detección de defectos.

Además, herramientas como JMeter para simular miles de pedidos simultáneos o Lookback para observar la interacción real de los usuarios, nos complementan para medir y asegurar esos atributos de calidad tan importantes, integrando las pruebas desde el primer día con Shift-Left, mitigando errores y costos. Sin duda adoptar las pruebas y las técnicas junto con las herramientas resulta ser clave para obtener y convertirse en una empresa fiable y con servicios de calidad.

REFERENCIAS BIBLIOGRÁFICAS

Fernández Riera, M. (2022). Puesta en producción segura: (1 ed.). Madrid, RA-MA Editorial. Recuperado de https://elibro.net/es/ereader/iacc/230285?page=65.

https://view.genially.com/651c300b68eb380011b08ce9

https://rise.articulate.com/share/UzncGAC54QXYw6DcsxeXL8ayJyhw9zyp#/lessons/ad4igz47E4_KcX2ORNUR9Ag24ECcZRgu

IBM (2023). Pruebas de rendimiento. https://www.ibm.com/docs/es/rtw/9.0.0?topic=phases-performance-testing